

ABSTRACT

This project presents an Automatic Student Attendance Management System that uses machine learning and face recognition technology to streamline and automate the process of student attendance. The system is designed with a multi-layered architecture that integrates Java Spring Boot for backend services, React for the frontend, and Python for face recognition.

The **face recognition module** is built using Python and utilizes machine learning algorithms to detect and recognize student faces in real-time using camera, ensuring a contactless and efficient method for attendance recording. Each recognized face is matched with the student's profile stored in the database. The backend, powered by **Java Spring Boot**, handles the business logic and database operations. It stores and manages student data, attendance records, and images in a secure and scalable **relational database** such as MySQL.

The system's frontend, developed with **React**, provides a user-friendly and responsive interface for administrators and students. It allows administrators to view and manage attendance records, register students, and update information, while students can view their attendance history. This project aims to eliminate traditional manual attendance processes, reduce human errors, and provide real-time access to attendance data.

By integrating **machine learning**, **database management**, and a **modern web interface**, this solution enhances the efficiency of attendance management in educational institutions.

CONTENTS

List of Figures Chapters	Page No
1.Introduction	
1.1 Overview	1
1.2 Objectives	1
1.3 Problem Formulation	1
1.4. Scope of the project	1
1.5 Feasibility	2
2.Requirements	
2.1 Requirements of the Project	3
2.2 Non functional requirements	3
2.3 Functional requirements	3
3.Analysis	
3.1 Use case Model	4
3.2 Activity Diagram	5
3.3 E-R Diagram	6
3.4 Component Design	7
3.5 System Architecture	8
4. Implementation	
4.1 Dataset	
4.1.1 Collection of Dataset	9
4.1.2 Testing Training Dataset	9
4.2 Code Snippet	
4.2.1 Face Recognition Module	11
4.2.2 Front End Module	15
4.2.3 Back End Module	41
5.User Screen	47
6.Testing	
1.Unit Testing	51
2.Integration Testing	52
3.System Testing	52
4.Performance Testing	52
5.Security Testing	53

7.Future Enhancements and conclusions

54

8.References

55

List of Figures

Figure Number	Figure Name	Page No
Figure 3.1	UML Diagram	4
Figure 3.2	Activity Diagram	5
Figure 3.3	ER Diagram	6
Figure 3.4	Component Design	7
Figure 3.5	System Architecture	8
Figure 4.1	Training Dataset	9
Figure 5.1	Login Page	47
Figure 5.2	Reset Password	47
Figure 5.3	Dashboard	48
Figure 5.4	Student Attendance Table	48
Figure 5.5	Attendance Report	49
Figure 5.6	Detail Reports	49
Figure 5.7	Downloaded Report	50

1.INTRODUCTION

1.1 OVERVIEW

The Automatic Student Attendance System utilizes machine learning and facial recognition technology to streamline the attendance process in educational institutions. By integrating a camera with face recognition capabilities, this system automates the attendance marking process. Students simply look into the camera, and the system captures their image, processes it using the LBPH (Local Binary Patterns Histogram) face recognition algorithm, and matches it with stored profiles in the database.

1.2 OBJECTIVE:

The main objective of this project is to **automate the attendance marking process** by utilizing facial recognition technology, making it more efficient and time-saving compared to traditional methods. By implementing **machine learning** techniques, the system aims to **enhance accuracy**, minimizing errors and ensuring that only authorized students are marked present. This approach significantly **reduces classroom interruptions** and frees up valuable time for both students and teachers

1.3 Problem Formulation

Traditional attendance methods in educational institutions, such as manual roll calls and ID card systems, are often time-consuming, error-prone, and vulnerable to manipulation. These approaches can lead to inaccuracies, proxy attendance, and significant classroom disruptions. As class sizes grow, the need for a more efficient and reliable solution becomes evident.

This project aims to address these issues by developing an Automatic Student Attendance System that utilizes facial recognition powered by machine learning. The system seeks to enhance accuracy, save time, and reduce administrative burdens while providing secure and real-time attendance tracking for both students and administrators.

1.4 Scope

The Automatic Student Attendance System will focus on several key areas to enhance the attendance management process in educational institutions. First, it will implement the LBPH (Local Binary Patterns Histogram) algorithm for accurate and automated attendance marking through facial recognition technology. A user-friendly web interface will be developed using React, allowing administrators to easily manage attendance records and generate reports. The system will utilize a MySQL database for real-time storage and updates of attendance data, ensuring accuracy and reliability.

1.5 Feasibility

The **Automatic Student Attendance System** demonstrates high feasibility across several key dimensions. **Technical feasibility** is strong due to the use of established technologies like the **LBPH (Local Binary Patterns Histogram)** algorithm for facial recognition, along with robust frameworks such as **Java Spring Boot** for the backend and **React** for the frontend. Existing libraries in Python, like OpenCV, facilitate the implementation, ensuring the system is technically sound. **Operationally**, the system is designed to streamline the current attendance process, requiring minimal changes to existing workflows. Training for administrators and students can enhance user adoption, making the system easy to implement.

2.REQUIREMENTS

2.1 System Requirements

2.1.1 Software Requirements:

Operating System: Windows, macOS, or Linux

Programming Language: Python 3, Java

Frame Works : Java Spring Boot , React

2.1.2 Hardware Requirements

GPU: NVIDIA GPU with CUDA support for efficient image generation

CPU: Multi-core processor (Intel i5 or equivalent)

RAM: 8 GB or higher

Storage: SSD with at least 10 GB of free space

2.2 Non functional Requirements

Performance: The system shall mark attendance within 5 seconds of capturing the student's image. The database and system must handle multiple concurrent users without performance degradation.

Usability: The user interface shall be intuitive and easy to use, requiring minimal training for students and administrators. The system shall provide clear error messages and prompts during interaction.

Scalability: The system should accommodate the growing number of students, with the ability to add new users and expand the database as needed. It should be easily expandable to integrate future features, such as mobile access.

2.3 Functional Requirements

User Authentication: Secure login for students and administrators with encrypted passwords.

Facial Recognition: Capture images for attendance marking using the LBPH algorithm for identity verification.

Attendance Management: Maintain attendance records in a MySQL database, allowing retrieval for specific dates and manual marking if recognition fails.

Reporting: Generate and export attendance reports in formats like Excel.

3.ANALYSIS

3.1 Use Case Model (UML Diagram)

The diagram shown is a **Use Case Diagram** for an **Automatic Attendance System**. It highlights two primary actors: **Student** and **Faculty**. The student interacts with the system for tasks like **Model Training** and **Face Recognition**. After their attendance is recorded, both students and faculty can access features like the **Dashboard** for viewing attendance data, **Current Status** for real-time updates, and **Reports Downloading** to retrieve attendance reports. This diagram outlines the core functionalities and interactions between users and the system.

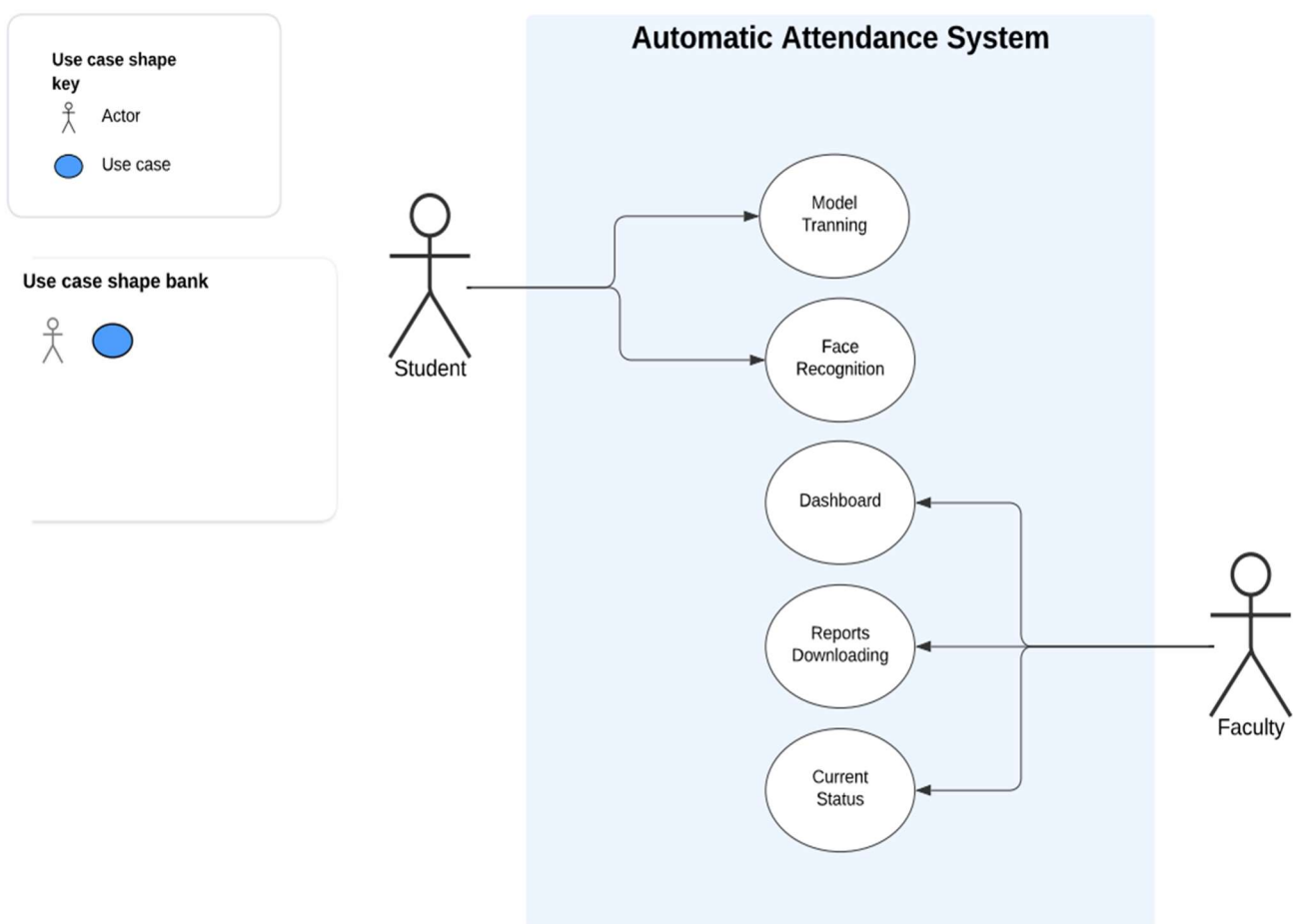


Figure 3.1

3.2 Activity Diagram

The **Activity Diagram** for the **Automatic Student Attendance System** outlines the process flow. Students look into the camera, which captures their image and processes it using the **LBPH Face Recognition Model**. If recognized, the system logs the student as "present" and updates the **Attendance Database**. If recognition fails, the system notifies the student.

For admins, the diagram includes logging in, viewing or retrieving attendance reports, and downloading data. After completing tasks, the admin logs out. This diagram visualizes the interactions between students and admins, from face recognition to attendance management and report generation.

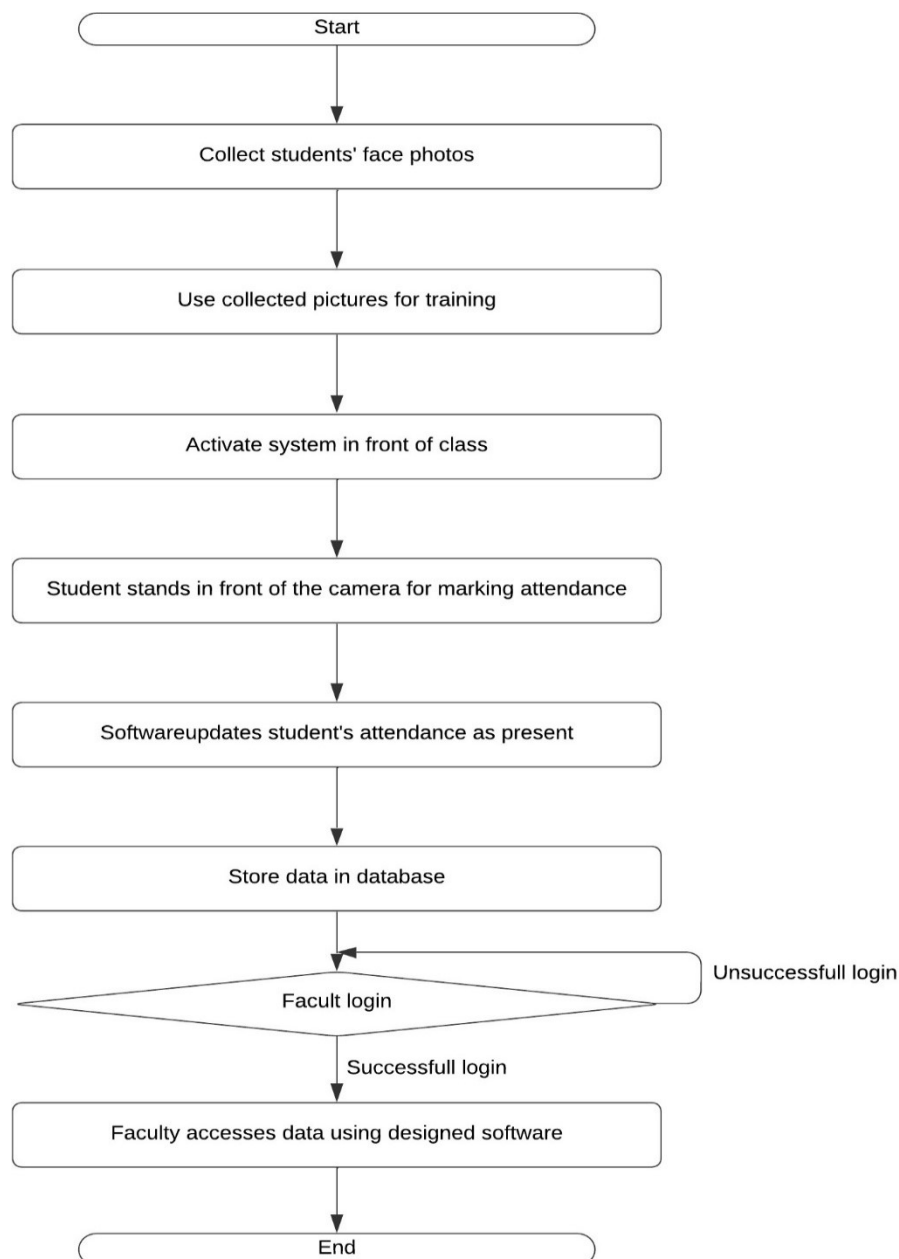


FIGURE 3.2

3.3 ER Diagram

The ER diagram for the Automatic Student Attendance System illustrates the relationships between five main entities: Student, Attendance, Admin, Report, and Camera. The Student entity holds essential information such as the student ID, name, roll number, department, and face encoding, which is used for face recognition. Each student is linked to multiple Attendance records, creating a one-to-many relationship. Additionally, each Student is connected to a single Camera in a one-to-one relationship, where the camera captures their face during attendance marking. The Attendance entity stores details like the attendance date, status (present or absent), and timestamp, and it is linked to the respective student. The admin entity manages the system by generating attendance reports and performing various administrative tasks. Each Admin can generate multiple Reports, establishing a one-to-many relationship between the two. The Report entity stores information about the type of report (e.g., daily, monthly) and the date it was generated. Finally, the Camera entity represents the device used for capturing students' faces, with each camera being linked to one student.

This diagram provides a clear overview of how attendance is managed, processed, and retrieved, with a structured database supporting efficient operations for students and administrators alike.

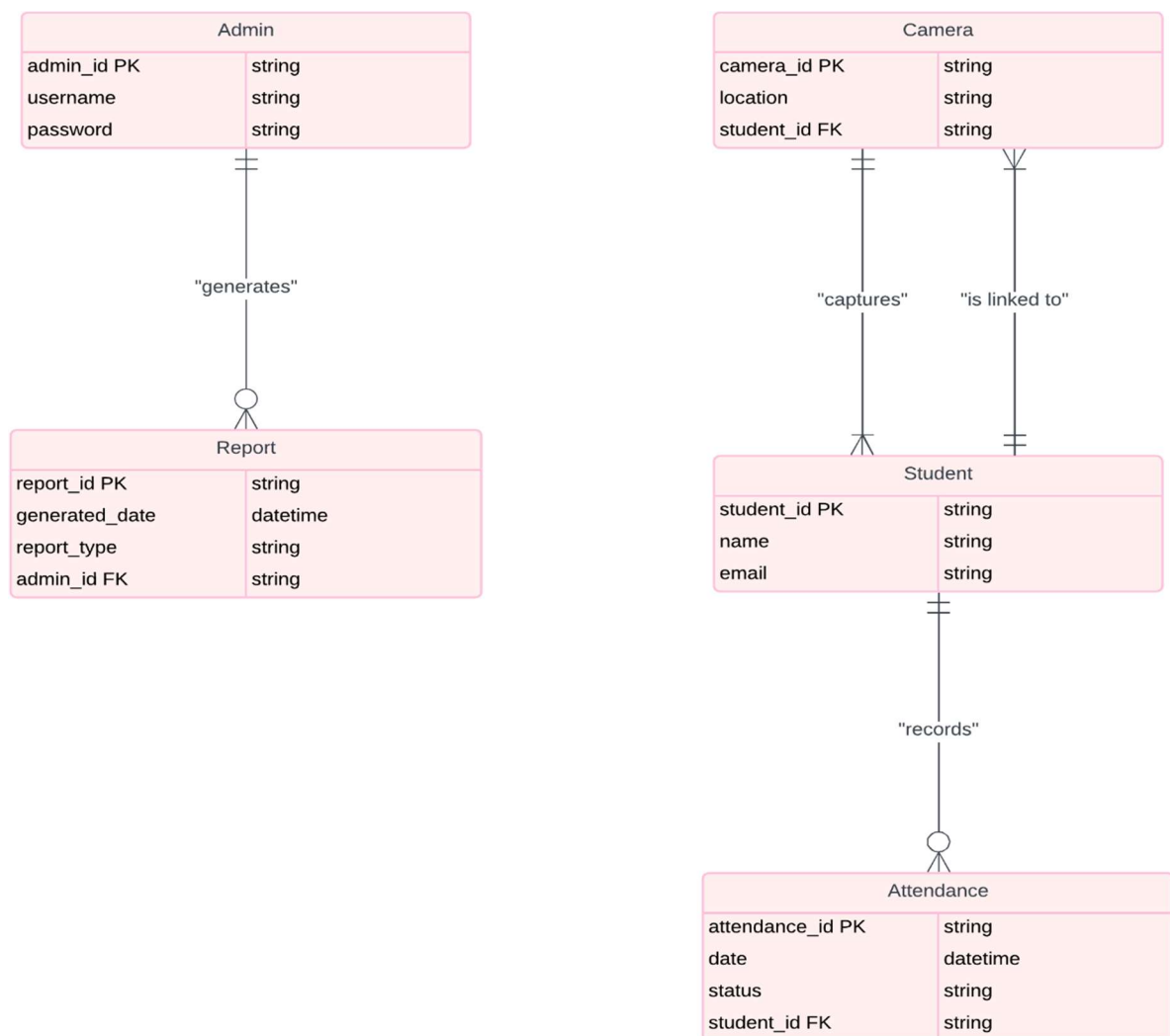


FIGURE 3.3

3.4 Component Design

Component Design in the context of the Automatic Attendance System focuses on defining the system's architecture by identifying and detailing the various components and their interactions. The system consists of a Frontend built with React, which includes key components such as the Login Page, Dashboard, and Report Viewer for user interaction and data presentation. The Backend, developed using Java Spring Boot, encompasses essential services like the Authentication Service for user verification, the Attendance Controller for processing attendance requests and managing the face recognition process, and the Report Generator for creating attendance reports. The Face Recognition Module, implemented in Python, is critical for capturing student images and recognizing faces through an LBPH model, working closely with the Attendance Updater to log attendance status. Finally, a MySQL Database stores vital information in structured tables for students, attendance records, and admin credentials. This component design ensures a clear separation of concerns, facilitating efficient communication between the various parts of the system and promoting scalability and maintainability

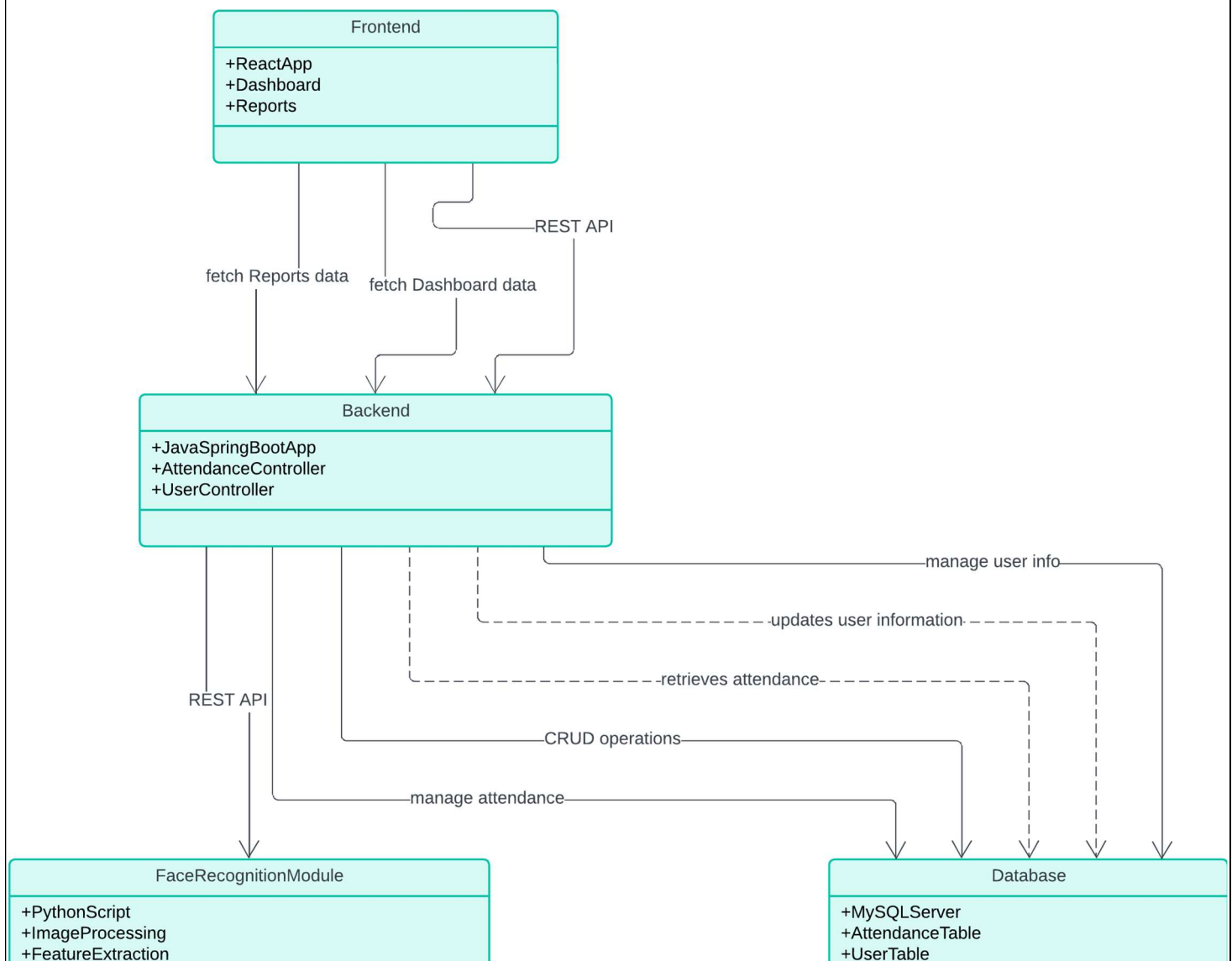


FIGURE 3.4

3.5 System Architecture

The System Architecture for the Automatic Attendance System is structured to ensure efficient user interaction and data management. At the user interface level, a React application provides features such as a Login Page for authentication, a Dashboard for attendance statistics, and a Report Viewer for accessing reports. The Application Layer utilizes Java Spring Boot, housing essential services like the Authentication Service for user sessions and the Attendance Service for processing attendance requests. The Face Recognition Module, implemented in Python, captures student images and recognizes faces using an LBPH model to log attendance accurately. Data is stored in a MySQL Database, which includes tables for student information, attendance records, and admin credentials. This architecture promotes modularity and scalability, delivering a robust solution for automating attendance management in educational settings.

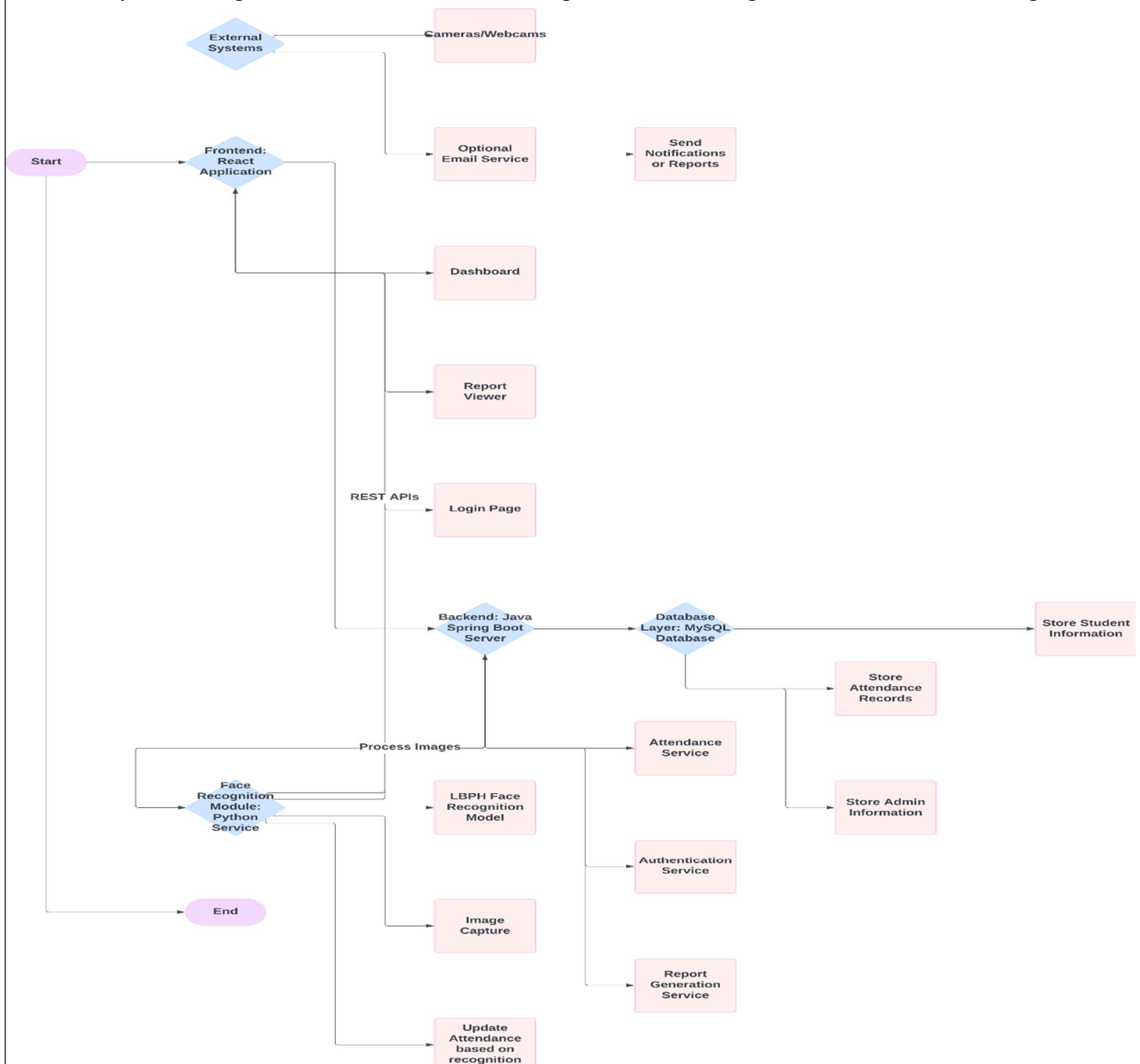


FIGURE 3.5

4.IMPLEMENTATION

4.1 Dataset

4.1.1 Training Dataset

The figure above shows the training dataset used for facial recognition and attendance marking. Each entry in the dataset is labeled systematically for easy identification during the training process. The dataset includes 200 entries, representing various expressions and orientations to improve model robustness. The data is raw and unprocessed, ensuring that the model can learn from realistic, varied conditions, which is crucial for accurate facial recognition and attendance tracking

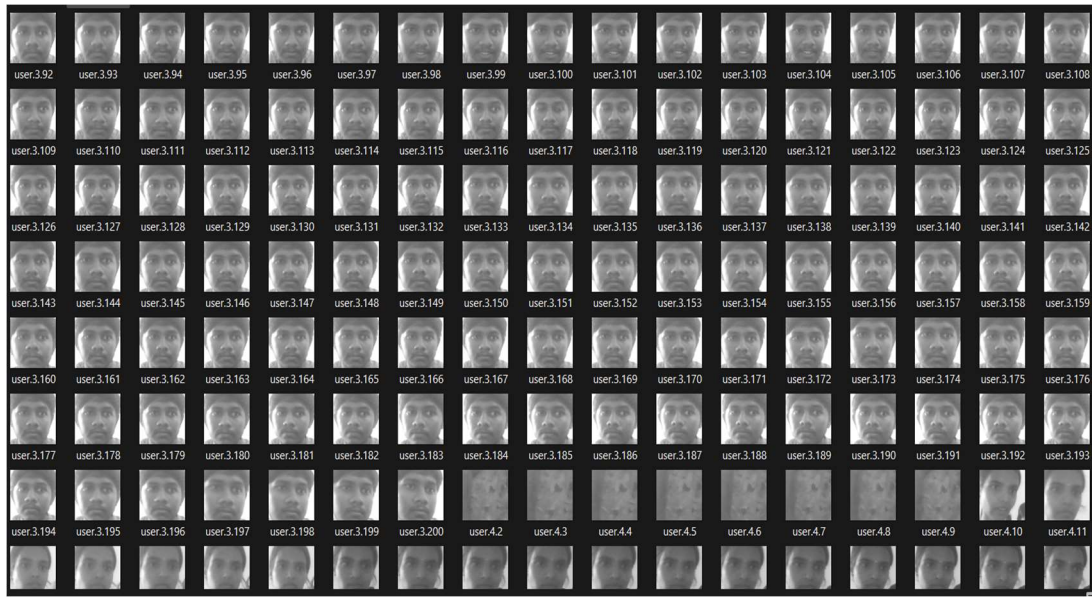


FIGURE 4.1

4.1.2 Testing Training Dataset

After training the facial recognition model, the dataset was tested to evaluate its performance in real-world scenarios. The testing dataset includes entries not used during the training process, ensuring that the model is evaluated on unseen data. This allows us to assess the model's accuracy in identifying faces and marking attendance under varied conditions, such as different facial expressions and lighting variations. The results from the testing phase help fine-tune the model for improved precision and reliability in actual applications.

4.2 Code Snippets

4.2.1 Face Recognition Module

```
import cv

def generate_dataset():

    face_classifier = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

def face_cropped(img):

    gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

    faces = face_classifier.detectMultiScale(gray, 1.3, 5)

    # scaling factor = 1.3

    # minimum neighbor = 5

    if faces is ():

        return None

    for (x,y,w,h) in faces:

        cropped_face = img[y:y+h,x:x+w]

        return cropped_face

cap = cv2.VideoCapture(0)

id =4

img_id = 1

while True:

    ret, frame = cap.read()

    if face_cropped(frame) is not None:

        img_id+=1

        face = cv2.resize(face_cropped(frame), (200,200))

        face = cv2.cvtColor(face, cv2.COLOR_BGR2GRAY)

        file_name_path = "data/user."+str(id)+"."+str(img_id)+".jpg"

        cv2.imwrite(file_name_path, face)
```

```

        cv2.putText(face, str(img_id), (50, 50), cv2.FONT_HERSHEY_COMPLEX, 1,
        (0, 255, 0), 2)

    cv2.imshow("Cropped face", face)

    if cv2.waitKey(1) == 13 or int(img_id) == 200: #13 is the ASCII character of Enter
        break

    cap.release()

    cv2.destroyAllWindows()

    print("Collecting samples is completed....")

generate_dataset()

```

4.2.2 Training of data

```

import os

import cv2

from PIL import Image #pip install pillow

import numpy as np # pip install numpy

def train_classifier(data_dir):

    path = [os.path.join(data_dir, f) for f in os.listdir(data_dir)]

    faces = []

    ids = []

    for image in path:

        img = Image.open(image).convert('L')

        imageNp = np.array(img, 'uint8')

        id = int(os.path.split(image)[1].split(".")[1])

        faces.append(imageNp)

        ids.append(id)

    ids = np.array(ids)

    # Train and save classifier

```

```

clf = cv2.face.LBPHFaceRecognizer_create()

clf.train(faces,ids)

clf.write("classifier.xml")

# print('completed')

train_classifier("data")

```

4.2.3 Face Recognition module

```

import cv2

import numpy as np

from PIL import Image

import pymysql

from datetime import datetime

def draw_boundary(img, classifier, scaleFactor, minNeighbors, color, text, clf, db_cursor,
detection_count):

    gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    features = classifier.detectMultiScale(gray_img, scaleFactor, minNeighbors, minSize=(30,
30))

    for (x, y, w, h) in features:

        cv2.rectangle(img, (x, y), (x+w, y+h), color, 2)

        face_img = gray_img[y:y+h, x:x+w]

        id, pred = clf.predict(face_img)

        confidence = int(100 * (1 - pred/300))

        if confidence > 80:

            if id == 2:

                name = "Rajasomeshwar"

            elif id == 3:

                name = "Swaraj"

```

```

elif id == 4:

    name = "Parameshwari"

elif id == 8:

    name = "Ganesh"

elif id == 9:

    name = "Parameshwari"

else:

    name = "UNKNOWN"

cv2.putText(img, name, (x, y-5), cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 1,
cv2.LINE_AA)

detection_count += 1

if detection_count >= 7:

img_pil = Image.fromarray(cv2.cvtColor(img, cv2.COLOR_BGR2RGB)) # Convert BGR to
RGB

img_byte_arr = cv2.imencode('.jpg', np.array(img_pil),
[int(cv2.IMWRITE_JPEG_QUALITY), 95])[1].tobytes()

attendance_datetime = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

sql = """INSERT INTO studentReal (studentId, NAME, PICTURE,
ATTENDANCE_DATETIME) VALUES (%s, %s, %s, %s) ON DUPLICATE KEY
UPDATE NAME = VALUES(NAME), PICTURE = VALUES(PICTURE),
ATTENDANCE_DATETIME = VALUES(ATTENDANCE_DATETIME)"""

db_cursor.execute(sql, (id, name, img_byte_arr, attendance_datetime))

return img, detection_count, True

else:

    cv2.putText(img, "UNKNOWN", (x, y-5), cv2.FONT_HERSHEY_SIMPLEX, 0.8,
(0, 0, 255), 1, cv2.LINE_AA)

    return img, detection_count, False

db_connection = pymysql.connect(

```

```

host="localhost",

user="root",

password="root",

database="world"

db_cursor = db_connection.cursor()

faceCascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")

clf = cv2.face.LBPHFaceRecognizer_create()

clf.read("classifier.xml")

video_capture = cv2.VideoCapture(0)

video_capture.set(cv2.CAP_PROP_FRAME_WIDTH, 640)

video_capture.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

frame_count = 0

detection_count = 0

stopped = False

while True:

    ret, img = video_capture.read()

    if not ret or stopped:

        break

    frame_count += 1

    if frame_count % 3 == 0:

        img, detection_count, stopped = draw_boundary(img, faceCascade, 1.1, 5, (255, 255, 255),
        "Face", clf, db_cursor, detection_count)

        cv2.imshow("Face Detection", img)

        if cv2.waitKey(1) == 13 or stopped:

            break

db_connection.commit()

```

```
db_cursor.close()

db_connection.close()

video_capture.release()

cv2.destroyAllWindows()
```

4.2.2 Front End Module

```
import { useState } from "react";

import { Link, useNavigate } from "react-router-dom";

import { useAuth } from "../security/AuthContext";

import { ToastContainer, toast } from "react-toastify";

import { useEffect } from "react";

import "../login.css";

import "react-toastify/dist/ReactToastify.css";

function LoginComponent() {

  const authContext = useAuth();

  const navigate = useNavigate();

  const [password, setPassword] = useState("");

  const [username, setUsername] = useState("");

  const [email, setEmail] = useState("");

  const [mobileNumber, setMobileNumber] = useState("");

  const [department, setDepartment] = useState("");

  const [rightPanelActive, setRightPanelActive] = useState(false);

  const handleSignUpClick = () => {

    setRightPanelActive(true);

  };

  const handleSignInClick = () => {

    setRightPanelActive(false);
```

```

};

useEffect(() => {

    if (authContext.verifyEmailMessageSuccess) {

        toast.success(authContext.verifyEmailMessageSuccess);

    }

}, [authContext.verifyEmailMessageSuccess]);

useEffect(() => {

    if (authContext.error) {

        toast.error(authContext.error);

    }

}, [authContext.error]);

useEffect(() => {

    if (authContext.success) {

        toast.success(authContext.success);

    }

}, [authContext.success]);

useEffect(() => {

    if (authContext.updatePasswordTokenSuccess) {

        toast.success(authContext.updatePasswordTokenSuccess);

    }

}, [authContext.updatePasswordTokenSuccess]);

const handleFormSubmit = async (event) => {

    event.preventDefault();

    const operation = event.target.operation.value;

    if (operation === "signup") {

        const data = {

```

```

        username: email,

        password: password,

        mobilenumber: "" + mobileNumber,

    data: {

        id: 0,

        name: username,

        joining_date: null,

        department: "hi",

        designation: null,

        teachingexperience: 0,

        industryexperience: 0,

        total_experience: 0,

    },

    };

    console.log(data);

    const signupSuccess = await authContext.signup(data);

    if (signupSuccess) {

        navigate(/verify);

        navigate(/login);

    } else {

    }

    } else if (operation === "login") {

        const loginSuccess = await authContext.login(email, password);

        if (loginSuccess) {

            console.log("login");

            navigate(/welcome);

```

```

    } else {

        // if (authContext.error) toast.error(authContext.error);

    }

}

};

return (

<div className="total-body">

    <div className="login-total">

        <div

            className={`login-container ${

                rightPanelActive ? "login-right-panel-active" : ""

            }}

            id="main"

        >

            <div className="login-sign-up">

                <form id="signupForm" method="post" onSubmit={handleFormSubmit}>

                    <h1>Create Account</h1>

                    <input type="hidden" name="facultyid" />

                    <input

                        type="text"

                        name="username"

                        placeholder="Username"

                        required

                        value={username}

                        onChange={(e) => setUsername(e.target.value)}

```

```
/>

<input

  type="email"

  name="email"

  placeholder="Email"

  required

  value={email}

  onChange={(e) => setEmail(e.target.value)}

/>

<input

  type="text"

  name="number"

  placeholder="Mobile Number"

  required

  value={mobileNumber}

  onChange={(e) => setMobileNumber(e.target.value)}

/>

<input

  list="browsers"

  name="department"

  placeholder="Department"

  value={department}

  onChange={(e) => setDepartment(e.target.value)}

/>

<datalist id="browsers">

  <option value="Information Technology" />
```

```

    <option value="Artificial Intelligence" />

    <option value="Data Science" />

    <option value="Computer Science Engineering" />

    <option value="Ece" />

</datalist>

<input

    type="password"

    name="password"

    placeholder="Password"

    required

    value={password}

    onChange={(e) => setPassword(e.target.value)}

/>

<input type="password" placeholder="Confirm Password" required />

<button type="submit">Sign-Up</button>

<input type="hidden" name="operation" value="signup" />

</form>

</div>

<div className="login-sign-in">

    <form id="signinForm" method="post" onSubmit={handleFormSubmit}>

        <h1>Sign In</h1>

        <input

            type="text"

            name="email"

            placeholder="Email"

            required

```

```

        value={email}

        onChange={(e) => setEmail(e.target.value)}

    />

    <input

        type="password"

        name="password"

        placeholder="Password"

        required

        value={password}

        onChange={(e) => setPassword(e.target.value)}

    />

    <input type="hidden" name="operation" value="login" />

    <button type="submit">Sign-In</button>

    <Link to="/forgetpassword">ForgetPassword?</Link>

</form>

</div>

<div className="login-overlay-container">

    <div className="login-overlay">

        <div className="login-overlay-left">

            <h1>Already have an</h1>

            <h1> Account</h1>

            <button id="signIn" onClick={handleSignInClick}>

                Sign In

            </button>

        </div>

        <div className="login-overlay-right">

```

```

    <h1>Create Account</h1>

    <button id="signUp" onClick={handleSignUpClick}>

        Sign Up

    </button>

</div>

</div>

</div>

</div>

<ToastContainer />

</div>

</div>

);
}

```

```
export default LoginComponent;
```

Dashboard

```

import React from "react";

import { Container, Row, Col, Card, Button } from "react-bootstrap";

import { useNavigate } from "react-router-dom";

import "../DashBoard.css"; // Import custom CSS for additional styles

const Dashboard = () => {

    const navigate = useNavigate(); // Hook for navigation

    // Static data for students' attendance summary

    const totalStudents = 30;

```

```

const presentStudents = 25;

const absentStudents = totalStudents - presentStudents;


// Function to handle button click and redirect to attendance page

const goToAttendancePage = () => {

  navigate("/attendance");

};


return (

  <Container fluid>

    {/* Welcome Heading */}

    <h2 className="mt-4 text-center welcome-title">Welcome Anand Babu!</h2>


    {/* Attendance Summary */}

    <Row className="mt-4">

      <Col md={4}>

        <Card className="summary-card bg-info text-white text-center">

          <Card.Body>

            <Card.Title>Total Students</Card.Title>

            <Card.Text>{totalStudents}</Card.Text>

          </Card.Body>

        </Card>

      </Col>

      <Col md={4}>

        <Card className="summary-card bg-success text-white text-center">

          <Card.Body>

```

```
<Card.Title>Students Present</Card.Title>
```

```
<Card.Text>{presentStudents}</Card.Text>
```

```
</Card.Body>
```

```
</Card>
```

```
</Col>
```

```
<Col md={4}>
```

```
<Card className="summary-card bg-danger text-white text-center">
```

```
<Card.Body>
```

```
<Card.Title>Students Absent</Card.Title>
```

```
<Card.Text>{absentStudents}</Card.Text>
```

```
</Card.Body>
```

```
</Card>
```

```
</Col>
```

```
</Row>
```

```
{/* Button to go to Attendance Page */}
```

```
<Row className="mt-4">
```

```
<Col className="text-center">
```

```
<Button variant="primary" size="lg" onClick={goToAttendancePage}>
```

```
View Attendance
```

```
</Button>
```

```
</Col>
```

```
</Row>
```

```
{/* Recent Activities */}
```

```
<Row className="mt-4">
```

```

<Col>

  <Card className="recent-activities-card shadow-lg">

    <Card.Header className="bg-secondary text-white">

      Recent Activities

    </Card.Header>

    <Card.Body>

      <Card.Text>

        <ul className="recent-activities-list">

          <li>Student John Doe marked present on August 24, 2024.</li>

          <li>

            Student Jane Smith requested leave on August 23, 2024.

          </li>

        </ul>

      </Card.Text>

    </Card.Body>

  </Card>

</Col>

</Row>

</Container>

);

};

```

```
export default Dashboard;
```

Student List

```

import React, { useState, useEffect } from "react";

import axios from "axios";

```

```

import "./attendanceList.css";

const AttendanceList = () => {

  const [attendanceData, setAttendanceData] = useState([]);

  const [loading, setLoading] = useState(true);

  useEffect(() => {

    // Fetch attendance data from the backend API

    const fetchAttendanceData = async () => {

      try {

        const response = await axios.get("/api/attendance"); // Replace with your API endpoint

        setAttendanceData(response.data);

      } catch (error) {

        console.error("Failed to fetch attendance data:", error);

      } finally {

        setLoading(false);

      }

    };

    fetchAttendanceData();

  }, []);

  const formatTime = (dateString) => {

    const date = new Date(dateString);

    return date.toLocaleTimeString();

  };

```

```

return (
  <div className="attendance-container">
    <h2>Student Attendance</h2>
    {loading ? (
      <p>Loading...</p>
    ) : attendanceData.length > 0 ? (
      <table className="attendance-table">
        <thead>
          <tr>
            <th>Student ID</th>
            <th>Name</th>
            <th>Entry Times</th>
            <th>Exit Times</th>
          </tr>
        </thead>
        <tbody>
          {attendanceData.map((student) => (
            <tr key={student.studentId}>
              <td>{student.studentId}</td>
              <td>{student.name}</td>
              <td>
                {student.entryTimes.length > 0
                  ? student.entryTimes.map((time, index) => (
                    <div key={index}>{formatTime(time)}</div>
                  ))
                }
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    ) : null}
  </div>
)

```

```

      : "No entries"}

    </td>

    <td>

      {student.exitTimes.length > 0

        ? student.exitTimes.map((time, index) => (

          <div key={index}>{formatTime(time)}</div>

        ))

        : "No exits"}

    </td>

  </tr>

  )})

</tbody>

</table>

) : (

  <p>No attendance data available.</p>

  )}

</div>

);

};

```

```
export default AttendanceList;
```

Welcome Page

```

import { useNavigate } from "react-router-dom";

import { useAuth } from "../security/AuthContext";

import AttendancePages from "../testing";

```

```

import { displayImages } from "../imagecheck";

function WelcomeComponent() {

  const navigate = useNavigate();

  const authContext = useAuth();


  console.log(authContext);

  return (

    <div className="WelcomeComponent">

      {/* <AttendancePages></AttendancePages> */}

      {/* <displayImages></displayImages> */}

      <h1>Welcome {authContext.username}</h1>

      Report Our Class-Addentence{" "}

      <button

        onClick={() => {

          navigate(/tables);

        }}

      >

        {" "}

        click here

      </button>

    </div>

  );

}

export default WelcomeComponent;

```

Detailed Data Display

```

import React, { useState, useEffect } from "react";

import "./studentReport.css";

import { ToastContainer, toast } from "react-toastify";

import "react-toastify/dist/ReactToastify.css";

import { useNavigate } from "react-router-dom";

import { useAuth } from "../../security/AuthContext";

import { CSVLink } from "react-csv";


// Helper function to calculate total stay time

const calculateTotalStayTime = (entryTimes, exitTimes) => {

  if (

    !entryTimes ||

    !exitTimes ||

    entryTimes.length === 0 ||

    exitTimes.length === 0

  ) {

    return [];

  }

  let totalStayTimes = [];

  for (let i = 0; i < Math.min(entryTimes.length, exitTimes.length); i++) {

    const entryTime = new Date(entryTimes[i]);

    const exitTime = new Date(exitTimes[i]);

    if (entryTime && exitTime) {

      const duration = exitTime - entryTime; // duration in milliseconds

```

```

const hours = Math.floor(duration / (1000 * 60 * 60));

const minutes = Math.floor((duration % (1000 * 60 * 60)) / (1000 * 60));

const seconds = Math.floor((duration % (1000 * 60)) / 1000);

totalStayTimes.push(`${hours}h ${minutes}m ${seconds}s);

}

}

return totalStayTimes;

};

const StudentReportDetailDepth = ({ date, option, count }) => {

const [rows, setRows] = useState([]);

const [loading, setLoading] = useState(false);

const [sortConfig, setSortConfig] = useState({

  key: null,

  direction: "ascending",

});

const [selectedEntryTimes, setSelectedEntryTimes] = useState([]);

const [selectedExitTimes, setSelectedExitTimes] = useState([]);

const [selectedTotalStayTimes, setSelectedTotalStayTimes] = useState([]);

const navigate = useNavigate();

const authContext = useAuth();

useEffect(() => {

  const getData = async () => {

    setLoading(true);

    try {

```

```
const data = await authContext.getAllStudentsByDateOptionDepth({  
  date,  
  option,  
});  
  
console.log("Fetched data:", data);  
  
if (!Array.isArray(data)) {  
  toast.error("Unexpected data format from the server.");  
  console.error("Unexpected data format:", data);  
  return;  
}  
  
if (data.length === 0) {  
  toast.info("No students found.");  
}  
  
setRows(data);  
} catch (error) {  
  console.error("Failed to fetch student data:", error);  
  toast.error("Failed to fetch student data.");  
} finally {  
  setLoading(false);  
}  
};
```

```

    getData();
  }, [count]));

const handleStudentIdClick = (studentId) => {
  navigate(/student/${studentId}/details);
};

const handleSort = (field) => {
  let direction = "ascending";

  if (sortConfig.key === field && sortConfig.direction === "ascending") {
    direction = "descending";
  }

  const sortedRows = [...rows].sort((a, b) => {
    if (typeof a[field] === "string") {
      return direction === "ascending"
        ? a[field].localeCompare(b[field])
        : b[field].localeCompare(a[field]);
    } else {
      return direction === "ascending"
        ? a[field] - b[field]
        : b[field] - a[field];
    }
  });

  setRows(sortedRows);

  setSortConfig({ key: field, direction });
};

```

```

const generateCSVData = () => {

  return rows.map(({ photoUrl, status, ...rest }) => rest);

};

const handleEditClick = () => {

  navigate("/edit");

};

const handleEntryCountClick = (entryTimes) => {

  setSelectedEntryTimes(entryTimes || []);

};

const handleExitCountClick = (exitTimes) => {

  setSelectedExitTimes(exitTimes || []);

  setSelectedTotalStayTimes(

    calculateTotalStayTime(selectedEntryTimes, exitTimes || [])

  );

};

const handleTotalStayClick = (totalStayTimes) => {

  setSelectedTotalStayTimes(totalStayTimes || []);

};

return (

  <div className="feedback-container">

```

```
<h2>Student Attendance Table</h2>
```

```
{loading ? (
```

```
<p>Loading...</p>
```

```
): rows.length > 0 ? (
```

```
<
```

```
<table className="feedback-table">
```

```
<thead>
```

```
<tr>
```

```
<th
```

```
onClick={() => handleSort("studentId")}
```

```
style={{ cursor: "pointer" }}
```

```
>
```

```
Student ID
```

```
{sortConfig.key === "studentId" &&
```

```
(sortConfig.direction === "ascending" ? " ▲ " : " ▼ ")}
```

```
</th>
```

```
<th
```

```
onClick={() => handleSort("name")}
```

```
style={{ cursor: "pointer" }}
```

```
>
```

```
Name
```

```
{sortConfig.key === "name" &&
```

```
(sortConfig.direction === "ascending" ? " ▲ " : " ▼ ")}
```

```
</th>
```

```
<th
```

```
onClick={() => handleSort("entryCount")}
```

```

        style={{ cursor: "pointer" }}
      >

      No. of Entries

      {sortConfig.key === "entryCount" &&

        (sortConfig.direction === "ascending" ? " ▲ " : " ▼ ")}

    </th>

    <th

      onClick={() => handleSort("exitCount")}

      style={{ cursor: "pointer" }}

    >

    No. of Exits

    {sortConfig.key === "exitCount" &&

      (sortConfig.direction === "ascending" ? " ▲ " : " ▼ ")}

    </th>

    <th

      onClick={() => handleSort("totalStayTime")}

      style={{ cursor: "pointer" }}

    >

    Total Stay Time

    {sortConfig.key === "totalStayTime" &&

      (sortConfig.direction === "ascending" ? " ▲ " : " ▼ ")}

    </th>

  </tr>

</thead>

<tbody>

  {rows.map((row) => (

```

```

<tr key={row.studentId}>

  <td>{row.studentId}</td>

  <td>

    <span

      onClick={() => handleStudentIdClick(row.studentId)}

      style={{ cursor: "pointer", color: "blue" }}

    >

      {row.name}

    </span>

  </td>

  <td>

    <span

      onClick={() => handleEntryCountClick(row.entryCount)}

      style={{ cursor: "pointer", color: "blue" }}

    >

      {row.entryCount ? row.entryCount.length : "No data"}

    </span>

  </td>

  <td>

    <span

      onClick={() => handleExitCountClick(row.exitCount)}

      style={{ cursor: "pointer", color: "blue" }}

    >

      {row.exitCount ? row.exitCount.length : "No data"}

    </span>

  </td>

```

```

<td>

  <span

    onClick={() => handleTotalStayClick(row.totalStayTime)}

    style={{ cursor: "pointer", color: "blue" }}

  >

    {row.totalStayTime ? row.totalStayTime.length : "No data"}

  </span>

</td>

</tr>

)}}

</tbody>

</table>

```

```

{selectedEntryTimes.length > 0 && (

  <div className="entry-times-details">

    <h3>Entry Times</h3>

    <ul>

      {selectedEntryTimes.map((time, index) => (

        <li key={index}>{time}</li>

      ))}

    </ul>

  </div>

)}

```

```

{selectedExitTimes.length > 0 && (

  <div className="exit-times-details">

```

```
<h3>Exit Times</h3>
```

```
<ul>
```

```
{selectedExitTimes.map((time, index) => (
```

```
<li key={index}>{time}</li>
```

```
))}
```

```
</ul>
```

```
</div>
```

```
}}
```

```
{selectedTotalStayTimes.length > 0 && (
```

```
<div className="total-stay-details">
```

```
<h3>Total Stay Times</h3>
```

```
<ul>
```

```
{selectedTotalStayTimes.length > 0 ? (
```

```
selectedTotalStayTimes.map((time, index) => (
```

```
<li key={index}>{time}</li>
```

```
))
```

```
): (
```

```
<li>No data available</li>
```

```
))
```

```
</ul>
```

```
</div>
```

```
}}
```

```
</>
```

```
): (
```

```
<p>No data available to display.</p>
```

```

    })

    <div className="actions-container">

      <CSVLink

        data={generateCSVData()}

        filename={"student-report.csv"}

        className="feedback-table-button"

        target="_blank"

      >

        Download Report

      </CSVLink>

      <button

        onClick={handleEditClick}

        style={{

          marginLeft: "10px",

          padding: "10px 20px",

          backgroundColor: "#007bff",

          color: "#fff",

          border: "none",

          borderRadius: "5px",

          cursor: "pointer",

        }}

      >

        Edit

      </button>

    </div>

    <ToastContainer

```

```

        position="bottom-right"

        autoClose={3000}

        hideProgressBar={false}

        newestOnTop

        closeOnClick

        rtl={false}

        pauseOnFocusLoss

        draggable

        pauseOnHover

    />

</div>

);

};

export default StudentReportDetailDepth;

```

4.2.3 Back End Module

AuthenticationController

```

package com.spring.spring_security_learn.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.spring.spring_security_learn.model.ApplicationUser;

```

```

import com.spring.spring_security_learn.model.LoginDTO;

import com.spring.spring_security_learn.model.LoginResponseDTO;

import com.spring.spring_security_learn.model.RegistrationDTO;

import com.spring.spring_security_learn.service.AuthenticationService;

import jakarta.servlet.http.HttpServletRequest;

@RestController

@RequestMapping("/auth")

@CrossOrigin("*")

public class AuthenticationController {

    @Autowired

    public AuthenticationController(AuthenticationService authenticationService) {

        super();

        this.authenticationService = authenticationService;

    }

    private AuthenticationService authenticationService;

    @PostMapping("/register")

    public String registerUser(@RequestBody RegistrationDTO body){

        System.out.println(body);

        return authenticationService.registerUser(body.getUsername(),
body.getPassword(),body.getMoblienumber(),body.getData());

    }

    @PostMapping("/login")

    public LoginResponseDTO loginUser(@RequestBody LoginDTO body){

        return authenticationService.loginUser(body.getEmail(), body.getPassword());

    }

}

```

DetailsController

```
package com.testingImage.LearnImageStore.Image;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import java.util.List;
import java.sql.Date;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Base64;
import java.util.stream.Collectors;

@RestController
@RequestMapping("/students")
@CrossOrigin("")

public class StudentController {

    @Autowired
    private StudentService studentService;

    @Autowired
    private StudentServiceDeep studentServiceDeep;

    @GetMapping("/s/{date}/{option}")
```

```

public ResponseEntity<List<StudentDTo>> getStudentPictures(@PathVariable LocalDate
date,@PathVariable String option) {

//    List<byte[]> images = studentService.getPicturesForStudentSpecfic(date,option);

List<StudentDTo> images = studentService.getPicturesForStudentSpecfic(date,option);

    HttpHeaders headers = new HttpHeaders();

    headers.set("Content-Type", "application/json"); // Set to JSON since you're returning a list

    return new ResponseEntity<>(images, headers, HttpStatus.OK);

}

```

```

@GetMapping("/{id}/pictures")

```

```

public ResponseEntity<List<StudentDTo>> getStudentPictures(@PathVariable Long id) {

    List<byte[]> images = studentService.getPicturesForStudent(id);

    List<StudentDTo> list=new ArrayList<>();

    HttpHeaders headers = new HttpHeaders();

    headers.set("Content-Type", "application/json"); // Set to JSON since you're returning a list

    for(int i=0;i<images.size();i++) {

        System.out.println(images.get(i));

        list.add(new StudentDTo(i,"c03","Rajsom","present",images.get(i), "itc"));

    }

    return new ResponseEntity<>(list, headers, HttpStatus.OK);

}

```

```

@GetMapping("/picturesAll")

```

```

public ResponseEntity<List<StudentDTo>> getStudentPicturesAll() {

    List<StudentDTo> images = studentService.getPicturesForStudentAll();

    HttpHeaders headers = new HttpHeaders();

    headers.set("Content-Type", "application/json"); // Set to JSON since you're returning a list

    return new ResponseEntity<>(images, headers, HttpStatus.OK);

}

```

```

    }

    @GetMapping("/d/{date}/{option}")

    public ResponseEntity<List<StudentInOutDetails>> getStudentPicturesDepth(@PathVariable LocalDate
date,@PathVariable String option) {

//    List<byte[]> images = studentService.getPicturesForStudentSpecfic(date,option);

    List<StudentInOutDetails> images =
studentServiceDeep.getPicturesForStudentSpecficInOutTime(date,option);

        HttpHeaders headers = new HttpHeaders();

        headers.set("Content-Type", "application/json"); // Set to JSON since you're returning a list


        return new ResponseEntity<>(images, headers, HttpStatus.OK);

    }

}

```

User Repository

```

package com.spring.spring_security_learn.repository;

import java.util.List;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.stereotype.Repository;

import com.spring.spring_security_learn.model.*;

@Repository

public interface UserRepository extends JpaRepository<ApplicationUser, Integer> {

    Optional<ApplicationUser> findByUsergmail(String username);

    boolean existsByUsergmail(String username);

    Optional<ApplicationUser> findById(Long userId);
}

```

```
}
```

StudentDetailsRepo

```
package com.testingImage.LearnImageStore.Image;

import org.springframework.data.jpa.repository.JpaRepository;

import org.springframework.data.jpa.repository.Query;

import org.springframework.data.repository.query.Param;

import org.springframework.stereotype.Repository;

import java.time.LocalDate;

import java.time.LocalDateTime;

import java.util.List;

import java.util.List;

@Repository

public interface StudentRepository extends JpaRepository<Student, Long> {

    List<Student> findByStudentid(Long studentId);

    @Query("SELECT s FROM Student s WHERE s.attendanceDatetime BETWEEN :startDate AND :endDate")

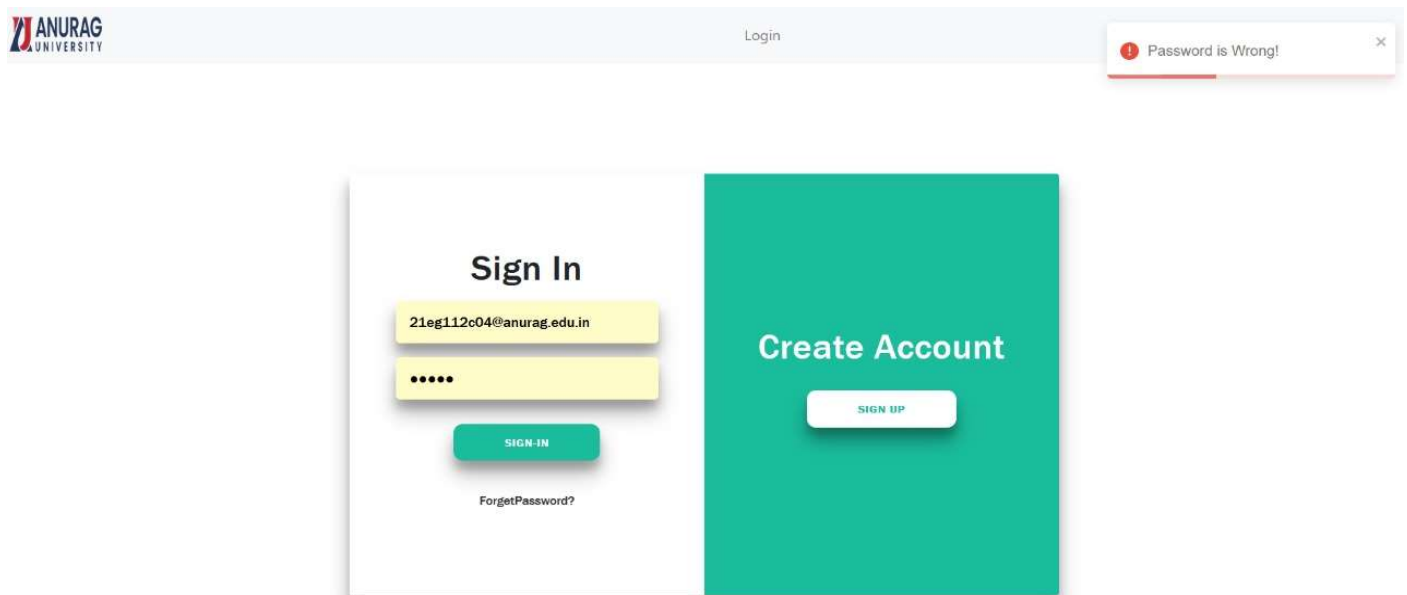
    List<Student> findByAttendanceDate(@Param("startDate") LocalDateTime startDate,

@Param("endDate") LocalDateTime endDate);

}
```

5 USER SCREEN

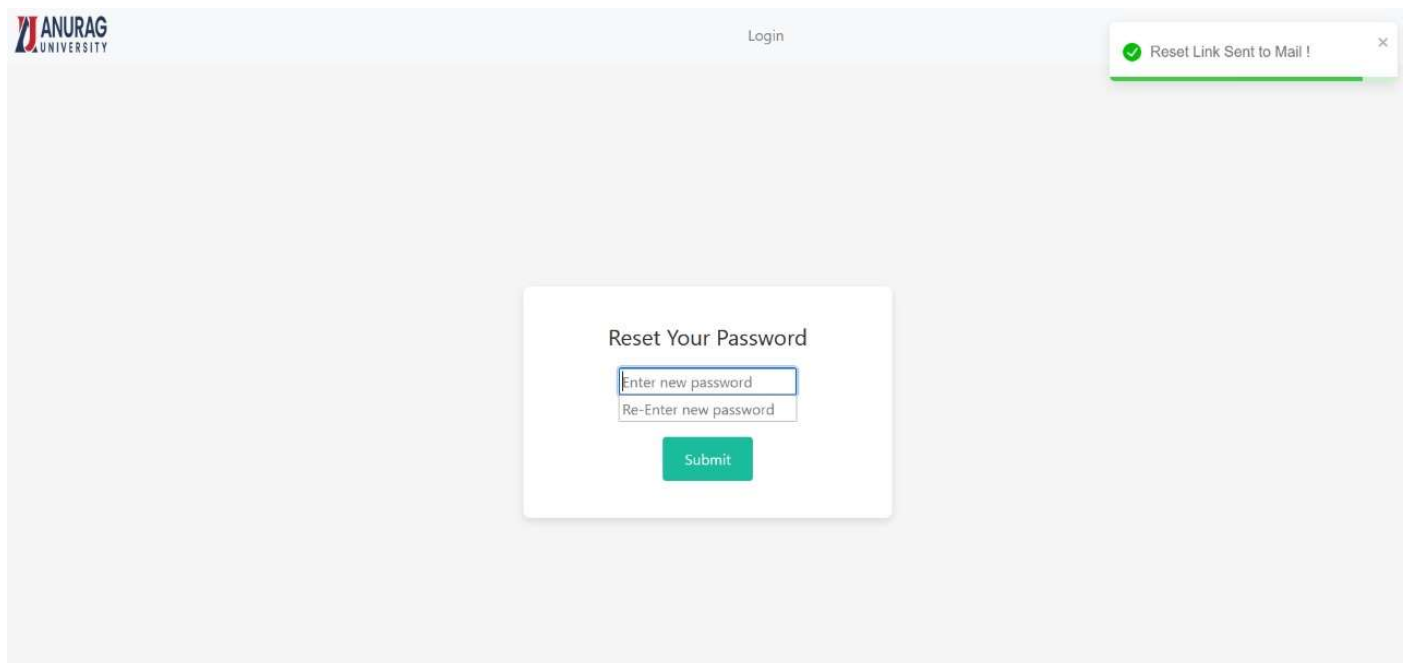
Login Page



The screenshot shows the login page of Anurag University. At the top left is the Anurag University logo. In the top center, the word "Login" is displayed. On the top right, a red error message box states "Password is Wrong!". The main content area is split into two panels. The left panel, titled "Sign In", contains a yellow input field with the email "21eg112c04@anurag.edu.in", another yellow input field with masked characters "*****", a green "SIGN-IN" button, and a link "ForgetPassword?". The right panel, titled "Create Account", features a green background and a white "SIGN UP" button.

FIGURE 5.1

Reset Password



The screenshot displays the "Reset Your Password" page. At the top left is the Anurag University logo. In the top center, the word "Login" is visible. On the top right, a green success message box says "Reset Link Sent to Mail !". The central form is titled "Reset Your Password" and includes two input fields: "Enter new password" and "Re-Enter new password". Below these fields is a green "Submit" button.

FIGURE 5.2

Dashboard

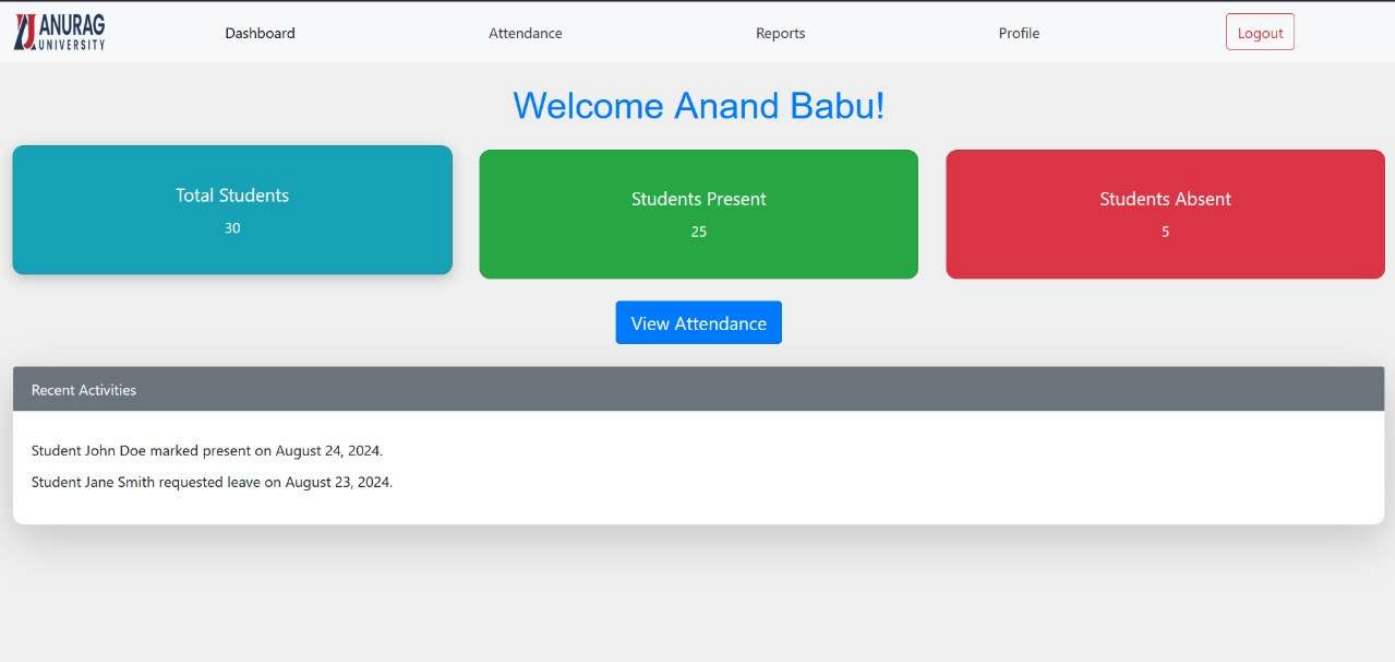


FIGURE 5.3

Student Attendance Table

Student ID	Name	Class	Status	Photo
3	Swaraj	it-c	present	
3	Swaraj	it-c	present	
4	Parameshwari	it-c	present	
4	Parameshwari	it-c	present	

FIGURE 5.4

Attendance Report

DashboardAttendanceReportsProfileLogout

Attendance Report

Date

21 / 09 / 2024

Choose an Option

Presents

Submit

Student Attendance Table

Student ID	Name	Class	Status	Photo
3	Swaraj	it-c	present	

FIGURE 5.5

Detail Reports

DashboardAttendanceReportsProfileLogout

Date: 21 / 09 / 2024

Choose an Option: Presents

Submit

Student Attendance Table

Student ID	Name	No. of Entries	No. of Exits	Total Stay Time
2	Rajasomeshwar	1	0	No data
3	Swaraj	1	1	No data
4	Parameshwari	1	1	No data

Entry Times

13:42:55

Download Report

Edit

FIGURE 5.6

Downloaded Report

	A	B	C	D	E
1	id	name	status	classN	studentId
2	0	Swaraj	present	it-c	3
3	1	Swaraj	present	it-c	3
4	2	Parameshv	present	it-c	4
5	3	Parameshv	present	it-c	4
6	4	Rajasomes	present	it-c	2
7					

FIGURE 5.7

6 TESTING METHODOLOGY

In this system we are using waterfall model to apply these ideas. In this each step is separated and when we finish one phase the output it is given input to the next phase. Also, backward approach can be implemented if there is a new requirement or to apply any update

1 Unit Testing:

Unit testing is essential to ensure that individual parts of your code work correctly.

Python - Face Recognition (LBPH Model):

- **Test Case 1: Detecting a Known Face**
 - **Description:** Ensure the system correctly recognizes and marks attendance for a known student.
 - **Expected Outcome:** The system identifies the student and returns their student_id.
- **Test Case 2: Detecting an Unknown Face**
 - **Description:** Test with a face not in the database.
 - **Expected Outcome:** The system should return an error or "unknown face" result.
- **Test Case 3: Image Quality**
 - **Description:** Provide a blurry or poor-quality image to the face recognition model.
 - **Expected Outcome:** The system should return "unrecognized" or prompt for a clearer image.

Spring Boot - Backend API:

- **Test Case 1: API for Fetching Attendance**
 - **Description:** Test the API to retrieve attendance by date or student.
 - **Expected Outcome:** The correct attendance data should be returned in JSON format.
- **Test Case 2: API for Adding Attendance**
 - **Description:** Test the API that adds attendance data after face recognition.
 - **Expected Outcome:** Attendance data should be added to the database successfully.
- **Test Case 3: Admin Login**
 - **Description:** Test the admin login functionality.
 - **Expected Outcome:** Admin should be able to log in with correct credentials.

React Frontend:

- **Test Case 1: Login Page**
 - **Description:** Test the login form for both students and admins.
 - **Expected Outcome:** Users should be able to log in, and proper error messages should be displayed for incorrect credentials.

- **Test Case 2: Attendance Page**

- **Description:** Test the page where students mark attendance through the camera.
- **Expected Outcome:** The page should open the camera and allow the student to submit their face for recognition.

- **Test Case 3: Admin Dashboard**

- **Description:** Test that all functionalities (view attendance, download reports) work properly.
- **Expected Outcome:** Admin should be able to retrieve and download data as required.

2. Integration Testing:

Check how different components of the system (frontend, backend, and database) work together.

- **Test Case 1: Face Recognition Integration**

- **Description:** Ensure that when a student submits their face, the system correctly processes it, recognizes the face, and updates the MySQL database.
- **Expected Outcome:** The attendance should be marked in the database for the recognized student.

- **Test Case 2: Admin Fetch Attendance Integration**

- **Description:** Check that when the admin requests attendance for a specific day, the backend fetches the data and the frontend displays it correctly.
- **Expected Outcome:** Accurate attendance data is shown in the admin panel.

3. System Testing:

Test the entire system from end to end.

- **Test Case 1: Full Attendance Workflow**

- **Description:** A student logs in, submits their face for recognition, and the system marks their attendance.
- **Expected Outcome:** The system should correctly identify the student and update the attendance in the MySQL database.

- **Test Case 2: Admin Generates Reports**

- **Description:** Test the report generation feature.
- **Expected Outcome:** The admin should be able to download the report in the specified format (CSV, PDF, etc.).

4. Performance Testing:

- **Test Case 1: Face Recognition Time**

- **Description:** Measure the time it takes for the system to recognize a face and mark attendance.

- **Expected Outcome:** The system should process each face within an acceptable time limit (e.g., 2-3 seconds).
- **Test Case 2: Load Testing for Admin Actions**
 - **Description:** Test how the system behaves when multiple admins are generating reports or fetching data simultaneously.
 - **Expected Outcome:** The system should handle concurrent admin actions without crashing or significant delays.

5. Security Testing:

- **Test Case 1: Unauthorized Access**
 - **Description:** Ensure that students and non-admin users cannot access admin functionalities.
 - **Expected Outcome:** Unauthorized access should be blocked with a proper error message.

7.FUTURE ENHANCEMENTS AND CONCLUSIONS

Future Enhancements:

In the future, the **Automatic Student Attendance System** can be improved by integrating additional biometric authentication methods like fingerprint scanning or iris recognition. This will increase both the security and accuracy of the attendance process, ensuring that no unauthorized individuals are marked present. Another valuable enhancement would be the development of a **mobile application** for students, allowing them to mark attendance remotely using their phone's camera for face recognition. This would make the system more accessible and user-friendly.

The current LBPH face recognition model could be upgraded to more advanced AI-based face detection and recognition algorithms, such as those built using deep learning frameworks like TensorFlow or PyTorch. These models offer higher accuracy and faster performance, improving the overall efficiency of the system. Additionally, an **offline mode** can be introduced, allowing attendance to be marked in areas with poor internet connectivity. The data can be stored locally and synced with the database once the connection is restored, making the system more reliable in varied environments.

An automated alert and notification system could also be integrated, allowing students and administrators to receive real-time notifications about attendance statuses, missed classes, or daily reports. Such enhancements would ensure the system is scalable, adaptable, and capable of meeting diverse institutional needs.

Conclusion:

The **Automatic Student Attendance System** is a powerful tool that leverages modern technology like **machine learning** and **biometric recognition** to streamline and automate attendance processes. With **Python** for face recognition, **Java Spring Boot** for backend hosting, and **React** for the frontend, the system effectively eliminates manual attendance, reduces errors, and provides a transparent and efficient attendance management solution.

Although the system is fully functional, there is significant potential for future improvements, such as adding more biometric options, mobile support, and enhanced AI models. As educational institutions continue to adopt digital solutions, systems like this one will play an essential role in ensuring accuracy, saving time, and offering a better overall user experience for students and administrators alike.

8 REFERENCES

1. OpenCV. (n.d.). Face Recognition with LBPH Model. OpenCV Documentation. Retrieved from https://docs.opencv.org/3.4/da/d60/tutorial_face_main.html
2. GeeksforGeeks. (2021, March 15). Face Recognition Using Python and OpenCV. GeeksforGeeks. Retrieved from <https://www.geeksforgeeks.org/face-recognition-using-python-opencv/>
3. Spring.io. (n.d.). Building RESTful APIs with Spring Boot. Spring Boot Guides. Retrieved from <https://spring.io/guides/gs/spring-boot/>
4. Baeldung. (2022, April 19). Spring Boot and MySQL Integration. Baeldung. Retrieved from <https://www.baeldung.com/spring-boot-mysql>
5. FreeCodeCamp. (2020, October 12). React Admin Dashboard Tutorial. FreeCodeCamp. Retrieved from <https://www.freecodecamp.org/news/react-admin-dashboard-template-tutorial/>
6. MySQL. (n.d.). MySQL Documentation. MySQL Official Website. Retrieved from <https://dev.mysql.com/doc/>
7. TensorFlow. (n.d.). TensorFlow for Face Recognition. TensorFlow Documentation. Retrieved from <https://www.tensorflow.org/>
8. Vogella. (2021, July 20). JUnit and Mockito Tutorial for Spring Boot Testing. Vogella. Retrieved from <https://www.vogella.com/tutorials/Mockito/article.html>
9. Baeldung. (2022, August 15). Unit Testing with Spring Boot. Baeldung. Retrieved from <https://www.baeldung.com/spring-boot-testing>
10. Towards Data Science. (2019, June 17). Deep Learning for Face Recognition. Retrieved from <https://towardsdatascience.com/face-recognition-how-lbph-works-90ec258c3d6b>